Teste de Performance 2 (TP2)

Respostas teóricas, diagramas e tabela comparativa

Desenvolvimento de Data-Driven Apps com Python
Turma 26E4_3

Aluno	Luis Claudio de Jesus
Disciplina	Desenvolvimento de Data-Driven Apps com Python
Professor	Tiago Xavier
Repositório no GitHub	https://github.com/luiscjesus-infnet/desenvolvimento_de_data-driven_apps_com_python_tp2.git
Questões respondidas neste documento	Parte 1: questões 3 e 4 Parte 2: questões 4, 5 e 6 Apêndice: diagramas de arquitetura das questões 1, 2 e 3 da Parte 2

As aplicações práticas (questões 1 e 2 da Parte 1 e questões 1, 2 e 3 da Parte 2) foram entregues como projetos separados, cada um com o seu próprio README.md com as instruções de execução.

Sumário

Seção	Conteúdo
Parte 1 - Questão 3	Limitações do modelo de tradução Helsinki-NLP/opus-mt-en-fr
Parte 1 - Questão 4	Limitações do modelo GPT-2 na geração de texto
Parte 2 - Questão 4	Limitações de usar LangChain para integrar a API da OpenAI
Parte 2 - Questão 5	Limitações de usar LangChain para integrar o modelo de tradução da HuggingFace
Parte 2 - Questão 6	Tabela comparativa entre FastAPI puro e LangChain
Apêndice A	Diagramas de arquitetura das aplicações (Parte 2, questões 1, 2 e 3)
Apêndice B	Referências e declaração de uso de ferramentas de IA

Observação sobre o contexto das respostas

Todas as respostas deste documento se referem às aplicações que eu mesmo implementei neste TP. Sempre que uma limitação teve efeito prático no meu código, eu indico onde ela aparece e como tratei (ou por que decidi não tratar). As quatro primeiras questões discursivas estão organizadas nos mesmos tópicos pedidos no enunciado.

Parte 1 - Questão 3

Limitações do modelo de tradução Helsinki-NLP/opus-mt-en-fr

A API da questão 2 da Parte 1 recebe um texto em inglês e devolve a tradução em francês usando o modelo **Helsinki-NLP/opus-mt-en-fr**, carregado localmente pela biblioteca *transformers*. É um modelo MarianMT (arquitetura Transformer com codificador e decodificador), com cerca de 74 milhões de parâmetros, treinado no corpus OPUS. É um modelo pequeno e dedicado a um único par de idiomas, e as limitações abaixo vêm bastante dessas duas características.

1. Limitações quanto à precisão da tradução

O modelo entrega um resultado bom em frase declarativa curta, de registro neutro e vocabulário comum, que é o tipo de texto predominante no corpus de treinamento. Fora disso a precisão cai, e os erros que observei ou que são previsíveis pela natureza do modelo são:

- **Ambiguidade lexical.** Palavras com mais de um sentido são resolvidas por estatística, não por compreensão. Em inglês, *bank* pode ser banco financeiro ou margem de rio; sem contexto suficiente o modelo escolhe a tradução mais frequente no treino, que pode ser a errada.
- **Gênero gramatical.** O francês marca gênero em substantivos e adjetivos; o inglês muitas vezes não. Em *the doctor arrived* o modelo precisa decidir entre *le médecin* e *la médecin*, e tende a escolher a forma mais frequente no corpus, geralmente a masculina. Isso é reprodução de viés dos dados, e não uma escolha informada.
- **Vocabulário de domínio.** Termos técnicos, jurídicos, médicos, siglas e nomes próprios aparecem pouco no treino. O modelo às vezes traduz literalmente um nome que deveria ser mantido, ou inventa um termo próximo.
- **Sem detecção de idioma.** O modelo é fixo em inglês para francês. Se o cliente enviar um texto em português, a minha API não reclama: ela devolve uma saída sem sentido, porque o modelo tenta traduzir de qualquer forma. Essa validação não existe na minha implementação e seria necessária em produção.
- **Perda de contexto entre frases.** Como o modelo tem limite de tokens por chamada, eu quebro o texto em frases no *services.py* e traduzo uma por uma. Isso permite traduzir o texto longo inteiro, mas cada frase é traduzida isolada. Em um par como *The company grew. It hired 50 people.*, o pronome da segunda frase perde a referência da primeira e pode sair com gênero errado. Foi uma troca consciente: prefiro traduzir o texto todo com alguma perda de coesão a truncar o texto.
- **Entrada com erros degrada a saída.** Erro de digitação e frase mal construída geram tokens raros e a tradução piora bastante.

2. Desafios de tempo de resposta e desempenho em grande escala

- **Geração token a token.** O decodificador gera um token por passo, de forma autorregressiva. O tempo de resposta cresce com o tamanho do texto de saída; não é uma operação de custo fixo.
- **Custo da primeira requisição.** Na primeira chamada o modelo precisa ser baixado (cerca de 300 MB) e carregado na memória. Para não pagar esse custo em cada requisição, guardo o pipeline em uma variável de módulo e carrego uma única vez. Mesmo assim, a primeira chamada depois de subir o servidor é visivelmente mais lenta que as seguintes.
- **CPU contra GPU.** Em CPU a inferência é viável porque o modelo é pequeno, mas cada requisição ocupa o processador. Por isso configurei o *device* para usar GPU quando houver. Um detalhe que aprendi: se o PyTorch instalado não for a versão com suporte a CUDA, a verificação devolve falso e tudo roda na CPU mesmo com placa de vídeo na máquina.
- **Escala horizontal cara.** Cada processo *worker* do uvicorn carrega a sua própria cópia do modelo na memória. Dobrar o número de workers dobra o consumo de memória, e não apenas a vazão. A escala não sai de graça.

- **Ausência de lote (batch).** A minha API trata uma requisição por vez. Em volume alto, o correto seria agrupar requisições em lote para aproveitar melhor a GPU, ou colocar o modelo atrás de um servidor de inferência dedicado, deixando o FastAPI apenas como porta de entrada.
- **Limite de tokens por chamada.** O modelo trabalha com no máximo 512 tokens por vez, o que me obrigou a tratar o texto longo na aplicação.

3. Restrições de custo e escalabilidade

Aqui a vantagem é clara e vale registrar: **não existe custo por chamada nem por token**. O modelo é aberto e roda na minha infraestrutura. O custo muda de natureza: deixa de ser variável e passa a ser fixo.

- **Infraestrutura.** Pago pelo servidor ligado, com memória e CPU suficientes, esteja ele atendendo mil requisições ou nenhuma. Se quiser GPU, o custo sobe bastante e a placa fica ociosa nos períodos de baixa demanda.
- **Memória por réplica.** Escalar significa replicar o modelo. Dez réplicas são dez cópias dos pesos na memória.
- **Um modelo por par de idiomas.** Esta é a restrição de escalabilidade mais importante deste modelo. O opus-mt-en-fr só faz inglês para francês. Para atender dez idiomas eu precisaria de dez modelos carregados, com dez vezes o consumo de memória e dez pipelines para manter. Um modelo grande multilíngue por API resolveria isso com uma única integração.
- **Manutenção.** O modelo está congelado no tempo. Não recebe atualização automática, e mantê-lo atualizado significa fazer o retreinamento ou o ajuste fino por conta própria, com dataset, hardware e tempo.
- **Dependências pesadas.** O torch e o transformers passam de 2 GB instalados, o que aumenta o tamanho da imagem de implantação e o tempo de subida de cada réplica.

4. Limitações na tradução de gírias, expressões idiomáticas e linguagem de contexto

Este é o ponto em que um modelo dedicado e pequeno fica mais atrás de um modelo grande de linguagem. O opus-mt faz correspondência estatística entre sequências; ele não tem conhecimento de mundo nem leitura de intenção.

- **Expressões idiomáticas.** Quando a expressão não está no corpus, a tradução sai literal e perde o sentido. *It is raining cats and dogs* deveria virar algo como *il pleut à verse*, e não uma frase sobre gatos e cachorros caindo do céu. O mesmo vale para *break a leg* e *piece of cake*.
- **Gírias e neologismos.** O corpus tem uma data de corte. Gíria de internet, termo novo e uso regional recente simplesmente não existem no vocabulário aprendido.
- **Registro e formalidade.** O francês distingue *tu* e *vous*; o inglês usa *you* para os dois. O modelo decide sozinho, geralmente pela forma mais frequente, e não tenho como instruí-lo a manter um tom informal, porque ele não aceita instrução.
- **Ironia, sarcasmo e humor.** Dependem de pragmática e de contexto extralinguístico, que estão fora do alcance do modelo.
- **Linguagem dependente de contexto externo.** Pronomes, elipses e referências a algo dito antes exigem memória do diálogo, que o modelo não tem. A minha quebra por frases agrava esse ponto.

Como eu mitigaria essas limitações

Em um cenário real, dependendo do que fosse mais crítico, eu combinaria: glossário de termos obrigatórios com pós-processamento da saída; ajuste fino do modelo com textos do domínio; troca por um modelo maior ou multilíngue quando a naturalidade importa mais que o custo; revisão humana nos textos sensíveis; e validação do idioma de entrada antes de chamar o modelo, que é a correção mais simples e que falta na minha implementação atual.

Parte 1 - Questão 4

Limitações do modelo GPT-2 no contexto da geração de texto

A API da questão 1 da Parte 1 recebe uma frase e devolve a continuação gerada pelo **GPT-2** (openai-community/gpt2), carregado localmente pela biblioteca *transformers*. É um modelo somente decodificador, com cerca de 124 milhões de parâmetros na versão base e janela de contexto de 1024 tokens, treinado para uma única tarefa: prever o próximo token. Entender isso explica quase todas as limitações a seguir.

1. A coerência do texto gerado

- **Não é um modelo de chat.** Esta é a limitação que mais chama atenção na prática. O GPT-2 completa texto, não conversa nem obedece instrução, porque não passou por ajuste por instruções nem por aprendizado com feedback humano. Se eu enviar *What is machine learning?*, ele pode continuar com outras perguntas parecidas em vez de responder. Em aula vimos isso acontecer: com o prompt *Hello*, ele gerou um texto solto em vez de cumprimentar de volta. Por isso o prompt mais adequado para esta API é uma frase a ser continuada, como *Artificial intelligence is a technology that*, e não uma pergunta.
- **Coerência local boa, global ruim.** A gramática e a sintaxe das frases saem corretas, o que dá uma impressão inicial muito boa. O problema aparece no texto mais longo: ele perde o fio do assunto, muda de tema, repete ideias e às vezes se contradiz dentro do mesmo parágrafo.
- **Janela de contexto curta.** Com 1024 tokens, o início de um texto longo sai da janela e deixa de influenciar a geração, o que piora a coerência exatamente onde ela é mais necessária.
- **Repetição.** Se eu desligasse a reamostragem (*do_sample=False*), o modelo escolheria sempre o token mais provável e entraria em laço, repetindo a mesma frase. Foi por isso que configurei *do_sample=True* com *top_k* e *top_p*: a aleatoriedade controlada melhora a leitura do texto.
- **Sem memória entre chamadas.** A minha API não guarda estado: cada requisição é independente. Não existe conversa, porque o modelo não recebe o histórico. Para ter memória eu precisaria de um componente à parte, como o que o LangChain oferece na Parte 2.

2. Possíveis falhas ou incoerências geradas por LLMs

A causa de fundo é sempre a mesma: o modelo atribui uma probabilidade a cada token do vocabulário e sorteia o próximo. Ele não verifica fato nenhum e não tem representação de verdade.

- **Alucinação.** É quando o modelo afirma algo plausível e falso. O exemplo usado no material da disciplina é bem didático: o modelo afirmou que o FastAPI usa Flask, quando na verdade o FastAPI usa Starlette. A frase está bem escrita e soa correta, e é justamente isso que torna o erro perigoso.
- **Invenção de dados concretos.** Datas, números, nomes de pessoas, citações e referências bibliográficas são inventados com naturalidade, porque são sequências estatisticamente prováveis.
- **Não sabe dizer que não sabe.** O modelo sempre gera alguma coisa. Não existe, na arquitetura, um caminho para recusar ou admitir ignorância.
- **Falta de reprodutibilidade.** Com *do_sample=True*, a mesma entrada gera saídas diferentes a cada chamada. Isso dificulta teste automatizado e auditoria. É o oposto do que busquei na tradução, onde usei temperatura zero justamente para ter previsibilidade.
- **Compromisso entre criatividade e erro.** Quanto maiores a temperatura e o *top_k*, maior a chance de o modelo escolher um token improvável: o texto fica mais variado e também mais sujeito a alucinação. Não existe configuração que resolva os dois lados; é uma escolha de projeto, e foi nesse ponto que calibrei os parâmetros no *services.py*.
- **Conhecimento desatualizado.** O treinamento do GPT-2 é de 2019. Nada posterior a isso existe para o modelo, e ele não tem como buscar informação externa por conta própria.

- **Risco maior sem humano no circuito.** Quando eu leio a saída, eu filtro o erro. Se a saída alimentar outro sistema automaticamente, o erro se propaga. Esse é o cenário mais delicado, porque uma alucinação passa a ser entrada de uma decisão.

3. Desempenho e questões de latência

- **Latência proporcional ao tamanho da saída.** A geração é autorregressiva: cada token exige uma passada pelo modelo. Dobrar *max_new_tokens* praticamente dobra o tempo de resposta. Foi por isso que expus esse parâmetro na API e limitei o valor máximo a 200, para o cliente não derrubar o tempo de resposta por acidente.
- **Custo quadrático da atenção.** O mecanismo de atenção cresce com o quadrado do tamanho da sequência, então prompt longo encarece a geração inteira, e não apenas o primeiro token.
- **Carga inicial.** O modelo tem cerca de 500 MB. Carrego uma única vez e guardo em variável de módulo, mas a primeira requisição continua mais lenta.
- **Concorrência limitada pelo hardware.** Deixei a função da rota sem *async* de propósito: a inferência é bloqueante, e assim o FastAPI a executa em uma thread separada sem travar o servidor. Isso evita travar o laço de eventos, mas não aumenta a capacidade real, que continua limitada pela CPU ou GPU disponível.
- **Escala com o mesmo problema do tradutor.** Cada worker carrega sua cópia do modelo, e sem agrupamento em lote a GPU fica subutilizada.
- **Comparação honesta.** Um modelo grande por API responde melhor e, muitas vezes, mais rápido que o GPT-2 rodando na minha máquina, porque usa hardware dedicado e otimizações de serviço. Em troca, entram o custo por token e a dependência de rede, que é exatamente o tema da questão 4 da Parte 2.

4. Limitações na geração de conteúdo apropriado

- **Não existe alinhamento.** O GPT-2 não passou por aprendizado com feedback humano nem tem filtro de segurança embutido. Ele não recusa pedido e não tem noção de política de uso: pode gerar conteúdo ofensivo, preconceituoso ou inadequado a partir de um prompt inocente, sem nenhum aviso.
- **Viés do corpus.** O treinamento usou páginas da web coletadas a partir de links do Reddit, predominantemente em inglês e de um público específico. Estereótipos de gênero, raça, religião e nacionalidade presentes nesses textos reaparecem nas continuações. Vale lembrar que a própria OpenAI liberou o GPT-2 em etapas, justamente por preocupação com uso indevido e geração de desinformação em escala.
- **Memorização.** Modelos de linguagem podem reproduzir trechos literais do material de treinamento, o que levanta questões de direito autoral e de dados pessoais.
- **Praticamente monolíngue.** O desempenho em português é muito ruim. Isso também apareceu em aula com outro modelo: por mais que se peça resposta em português no prompt do sistema, o modelo responde em inglês, porque não foi treinado para isso. Na minha API eu assumo prompts em inglês.
- **O que faltaria para produção.** A minha aplicação é uma demonstração e não tem nenhuma dessas proteções. Em produção eu precisaria de filtro de conteúdo na entrada e na saída, lista de termos bloqueados, registro das gerações para auditoria, limite de requisições por usuário e um aviso explícito de que o texto foi gerado por inteligência artificial.

Conclusão prática. O GPT-2 é excelente para aprender como uma aplicação com modelo de linguagem funciona, porque é pequeno, aberto e roda em máquina comum. Mas a saída dele não deve ser tratada como fato nem exposta a usuário final sem revisão.

Parte 2 - Questão 4

Limitações de utilizar LangChain para integrar a API da OpenAI

A aplicação da questão 2 da Parte 2 traduz do inglês para o francês usando um modelo da OpenAI chamado pelo LangChain, com a classe **ChatOpenAI** encadeada em *PromptTemplate* | *modelo* | *StrOutputParser*. A diferença central em relação à Parte 1 é que a inferência deixou de acontecer na minha máquina e passou a ser um serviço de terceiro acessado por rede. Praticamente todas as limitações abaixo derivam disso.

1. Latência de resposta

- **A rede entra na conta.** O tempo de resposta agora é a soma de: ida e volta na rede com negociação TLS, tempo de fila no provedor, geração dos tokens e o processamento local do prompt e do parser. Na Parte 1 só existia o terceiro item.
- **A geração continua autorregressiva.** Resposta longa custa mais tempo, exatamente como no modelo local. Modelo maior tende a gerar cada token mais devagar, mesmo rodando em hardware melhor.
- **Latência pouco previsível.** Eu não controlo a carga do provedor. A média pode ser boa e os percentis altos serem bem piores, com variação ao longo do dia. O modelo local é mais lento em média, mas mais estável e previsível, o que para alguns sistemas importa mais.
- **Sobrecusto das camadas do LangChain.** Montar o prompt, validar e converter a saída acrescenta alguns milissegundos. Isso é irrelevante perto do tempo de rede em uma chamada única, mas deixa de ser irrelevante em uma cadeia com vários passos sequenciais, porque cada passo é uma nova chamada remota e as latências se somam.
- **O que fiz e o que faria.** Monto a cadeia e o cliente uma única vez, para não recriar a conexão em cada requisição. Em produção eu acrescentaria resposta em streaming, cache das traduções repetidas (que funciona bem com temperatura zero), limite de tokens de saída e chamadas assíncronas quando houver vários textos independentes para traduzir.

2. Limites de uso da API da OpenAI

- **Limite de taxa.** Existe um teto de requisições por minuto e de tokens por minuto. Ao estourar, a API devolve o código HTTP 429. No plano gratuito que usei esse teto é baixo, e é fácil atingi-lo só testando a aplicação.
- **Tratamento de 429 é obrigatório.** Sem repetição com espera progressiva (*retry* com *backoff*), a aplicação simplesmente falha sob carga. A minha implementação devolve HTTP 503 quando a chamada não dá certo, o que é honesto com o cliente, mas um sistema de produção deveria tentar de novo antes de desistir, e idealmente enfileirar a requisição.
- **Limite de contexto e cota.** Há um máximo de tokens por requisição e normalmente também uma cota diária ou mensal, o que limita o tamanho do texto que posso traduzir de uma vez.
- **Gestão da chave de API.** A chave é um segredo e não pode ir para o repositório. Eu a leio de variável de ambiente e o *.gitignore* ignora arquivos *.env*. Se a chave vazar, o prejuízo é direto, porque alguém consome a minha cota e, em plano pago, a minha fatura.
- **Dependência de terceiro.** Se o provedor ficar indisponível, minha aplicação para, e não há nada que eu possa fazer no meu código. Modelos também são depreciados: o identificador que uso hoje pode deixar de existir, e a atualização é obrigatória e fora do meu controle.
- **Privacidade e termos de uso.** O texto do usuário sai da minha infraestrutura e vai para um serviço externo. Para dado pessoal ou sigiloso isso precisa ser avaliado com cuidado, inclusive do ponto de vista da LGPD. Nesse cenário o modelo local da Parte 1 tem uma vantagem que não é técnica, e sim de conformidade.
- **Observação sobre a implementação.** Usei o modelo da OpenAI pela API compatível da Groq, que tem plano gratuito, como foi mostrado em aula. Como a classe *ChatOpenAI* é a mesma nos dois casos, apontar para a API oficial da OpenAI exige apenas trocar a chave e a URL base, sem alterar o código.

Isso ilustra bem a vantagem da interface padronizada, e também que os limites de uso dependem do provedor contratado, e não do meu código.

3. Desafios de escalabilidade e custo

É importante separar as duas coisas, porque elas vão em direções opostas em relação à Parte 1.

- **Escalabilidade de infraestrutura: muito melhor.** A minha API fica leve, sem modelo na memória e sem precisar de GPU. Cada instância consome pouco, sobe rápido e escala horizontalmente sem dificuldade, porque o peso da inferência está no provedor. Comparando com a Parte 1, deixei de ter o problema de carregar uma cópia do modelo por worker.
- **Escalabilidade passa a ser contratual.** Em troca, o teto deixa de ser o meu hardware e passa a ser a cota e o orçamento. Não resolvo um gargalo comprando mais memória: preciso negociar limite maior com o provedor.
- **Estrutura de custo invertida.** No modelo local o custo é praticamente todo fixo (servidor ligado) e o custo marginal por requisição é próximo de zero. Na API é o contrário: custo fixo próximo de zero quando ninguém usa, e custo variável por token de entrada e de saída que cresce de forma linear com o uso.
- **Análise objetiva do ponto de equilíbrio.** Dando nome às variáveis, o custo total do modelo local é aproximadamente *Cfixo*, e o custo total da API é aproximadamente *n x Cchamada*, onde *n* é o número de requisições no período. Há um volume *n* = Cfixo / Cchamada* em que os dois se igualam: abaixo dele a API sai mais barata, porque eu não pago servidor ocioso; acima dele hospedar o próprio modelo compensa. Para uma API de tradução com volume baixo ou irregular, a API comercial quase sempre ganha. Para traduzir um catálogo com milhões de frases, o modelo local dedicado ganha com folga. Não informo valores porque os preços mudam com frequência e dependem do contrato, mas a forma da comparação é essa.
- **Custo imprevisível no total.** O custo por chamada é conhecido, mas o custo do mês depende do volume, que pode variar muito. Isso exige monitoramento de consumo, limite de gasto configurado no provedor, limite de tamanho da entrada e cache, senão um pico de uso vira um susto na fatura.

4. Qualidade das traduções em comparação com outros modelos

Comparando diretamente com o opus-mt-en-fr da Parte 1, que tem cerca de 74 milhões de parâmetros, o modelo usado aqui tem cerca de 20 bilhões. A diferença aparece no resultado, e em aula ficou visível que o modelo maior responde de forma bem mais completa e coerente.

Onde o modelo grande por API é melhor:

- Lida muito melhor com expressão idiomática, gíria e ambiguidade, porque tem contexto e conhecimento de mundo muito mais amplos.
- Mantém coesão ao longo de um texto com várias frases, resolvendo pronome e referência entre frases. Não preciso quebrar o texto em frases como fiz na Parte 1.
- Aceita instrução: posso pedir que mantenha o tom formal, que preserve nomes próprios ou que respeite um glossário, direto no prompt. Isso simplesmente não é possível com o opus-mt.
- Vai bem em texto de domínio técnico, onde o modelo pequeno erra termos.

Onde o modelo dedicado leva vantagem:

- **Não é especializado em tradução.** O modelo grande às vezes faz mais do que foi pedido: adiciona comentário, explica a escolha ou reformula o texto. Precisei instruir no prompt a responder apenas com a tradução, exatamente para evitar que a resposta sujasse o JSON da minha API.
- **Risco de injeção de prompt.** Como o modelo entende instrução, um texto de usuário contendo algo como *ignore as instruções anteriores e faça outra coisa* pode desviar o comportamento. O opus-mt não corre esse risco, porque não interpreta instrução: ele apenas traduz. Esse é um ponto de segurança concreto em favor do modelo dedicado quando a entrada vem de terceiros.

- **Saída não determinística por natureza.** Reduzi isso com temperatura zero, mas a garantia não é a mesma de um modelo de tradução dedicado.
- **Custo e latência por chamada.** O modelo pequeno, depois de carregado, é barato e rápido para frase curta em volume alto.

Conclusão. Não existe escolha melhor em termos absolutos. Para tradução de frase curta, em grande volume, com orçamento apertado ou com exigência de privacidade, o modelo dedicado local atende melhor. Para texto que depende de contexto, naturalidade e flexibilidade de instrução, o modelo grande por API entrega qualidade bem superior. O ganho de usar LangChain é justamente poder trocar entre os dois sem reescrever a aplicação.

Parte 2 - Questão 5

Limitações de usar LangChain para integrar o modelo de tradução da HuggingFace

A aplicação da questão 3 da Parte 2 traduz do inglês para o alemão com o modelo **Helsinki-NLP/opus-mt-en-de**, embrulhado na classe **HuggingFacePipeline** do LangChain. É o cenário misto: o modelo roda localmente, como na Parte 1, e a chamada é padronizada pelo LangChain, como na questão anterior. Isso significa que eu herdo as limitações do modelo local e ainda acrescento uma camada de framework.

1. Desempenho e tempo de resposta

- **O LangChain não acelera nada.** Quem executa a inferência é o mesmo pipeline do *transformers* da Parte 1. O LangChain é uma camada de orquestração, não de otimização. Todo o custo de inferência local continua igual, inclusive a dependência de CPU ou GPU.
- **Primeira chamada lenta.** Download do modelo (cerca de 300 MB) mais carga na memória. Guardo a cadeia montada em variável de módulo para pagar esse custo uma única vez.
- **Camada extra no caminho.** Cada requisição passa por *PromptTemplate*, depois *HuggingFacePipeline* e depois *StrOutputParser*, em vez de chamar o pipeline diretamente. O custo disso é pequeno diante da inferência, mas existe, e fica proporcionalmente mais visível em tradução de frase muito curta, onde a inferência é rápida.
- **Chamada síncrona e bloqueante.** Mantive a rota sem *async* para o FastAPI executar a inferência em uma thread separada. Ainda assim, o número de traduções simultâneas é limitado pelo processador.
- **Sem agrupamento em lote.** O *HuggingFacePipeline* aceita *batch_size*, mas a minha API trata uma requisição por vez, então esse recurso não é aproveitado. Em volume alto seria um dos primeiros pontos a mudar.
- **Texto longo.** Diferente da questão 2 da Parte 1, aqui mando o texto em uma única chamada para manter o código simples e focado na integração. Como o modelo tem limite de tokens, texto muito grande é truncado. Foi uma decisão consciente de escopo, e não um descuido.

2. Consumo de recursos computacionais

- **Dependências pesadas.** Este é o projeto com mais dependências dos cinco: FastAPI, transformers, torch, sentencepiece e os três pacotes do LangChain. Somando, o ambiente virtual passa facilmente de 2 GB, o que aumenta o tamanho da imagem de implantação e o tempo de subida.
- **Memória.** Pesos do modelo, mais o runtime do PyTorch, mais os buffers de cada requisição. E cada worker do uvicorn carrega a sua própria cópia, então a memória se multiplica com a escala. A solução por API da questão anterior não tem nada disso.
- **CPU e GPU.** A inferência usa várias threads e disputa processador com o resto da aplicação. A GPU resolve, mas exige a versão do PyTorch compilada com CUDA: sem ela, a verificação de disponibilidade devolve falso e tudo roda na CPU mesmo havendo placa de vídeo instalada. Esse detalhe apareceu em aula e é uma pegadinha fácil de passar despercebida.
- **Disco.** O cache de modelos da HuggingFace cresce a cada modelo baixado, e não é limpo automaticamente.
- **Multiplicação por par de idiomas.** Como na Parte 1, cada par de idiomas é um modelo separado. Oferecer inglês para alemão e inglês para francês na mesma aplicação significa dois modelos carregados ao mesmo tempo.

3. Possíveis limitações no ajuste fino do modelo

- **O LangChain não faz ajuste fino.** Esta é a limitação mais importante deste tópico. O framework consome um pipeline que já existe; ele não treina nem ajusta modelo. Para fazer ajuste fino eu tenho que sair do LangChain, treinar com as ferramentas do *transformers*, salvar o modelo resultante e só depois

embrulhar esse modelo no *HuggingFacePipeline*. O LangChain atua no uso do modelo, e não no ciclo de treinamento.

- **Controle de parâmetros fica indireto.** Na Parte 1 eu passava *generation_config* e *clean_up_tokenization_spaces* diretamente na chamada do pipeline. Com o LangChain, esses ajustes precisam ir por *pipeline_kwargs* no *invoke* ou ser definidos na criação do pipeline. Funciona, mas é menos óbvio e um pouco mais escondido na documentação, o que atrapalha quem está começando.
- **Engenharia de prompt quase não funciona aqui.** Este foi o ponto que mais me chamou atenção ao implementar. O opus-mt não é um modelo de instrução: ele traduz tudo o que recebe. Se eu escrevesse *Translate to German:* no template, essa instrução seria traduzida para o alemão junto com a frase do usuário e apareceria na resposta. Por isso o meu template contém apenas *{texto}*. Na prática, o recurso mais forte que o LangChain oferece, que é a composição de prompts, fica praticamente inutilizado com um modelo de tradução dedicado. Todo o ajuste de comportamento vem da escolha do modelo e dos parâmetros de geração.
- **Ajuste fino real tem custo alto.** Exige dataset paralelo do domínio (pares inglês-alemão), GPU, tempo de treinamento e avaliação. E, uma vez feito, gera um modelo próprio que eu passo a ter que versionar, armazenar e manter.
- **Alternativas mais baratas.** Antes de partir para ajuste fino, vale testar outro modelo do Hub, aplicar glossário e pós-processamento na saída, ou trocar para um modelo grande por API, que aceita instrução e dispensa treinamento.

4. Comparação com o uso direto da biblioteca da HuggingFace

A comparação abaixo é entre a questão 2 da Parte 1 (transformers chamado diretamente) e esta questão 3 da Parte 2 (o mesmo tipo de modelo via LangChain).

Aspecto	HuggingFace direto (Parte 1)	Com LangChain (Parte 2, Q3)
Dependências	FastAPI, transformers, torch e sentencepiece.	As mesmas, mais langchain, langchain-core e langchain-huggingface.
Desempenho da inferência	É o próprio pipeline executando. Nenhuma camada no meio.	Idêntico, pois quem executa é o mesmo pipeline. Há um pequeno sobrecusto das camadas de prompt e parser.
Controle dos parâmetros	Direto e explícito na chamada do pipeline.	Indireto, via pipeline_kwargs ou na criação do pipeline. Menos óbvio.
Troca de modelo ou de provedor	Exige reescrever o serviço, porque cada biblioteca tem a sua própria forma de chamada.	Troca de uma linha. Fake LLM, modelo local e API são chamados todos por invoke, com a mesma interface.
Encadeamento de etapas	Feito na mão, com o meu próprio código de cola.	Nativo: operador de composição, parsers, memória, ferramentas e agentes.
Depuração	Pilha de erro curta. O erro vem do transformers e é fácil de localizar.	Pilha mais profunda. O erro atravessa as camadas do framework antes de chegar ao meu código.
Curva de aprendizado	Preciso conhecer apenas o transformers.	Preciso conhecer o transformers e também os conceitos do LangChain.
Estabilidade da API do framework	Depende apenas das versões do transformers.	Depende de três pacotes do LangChain, que evoluem rápido. Código escrito hoje pode precisar de ajuste em poucos meses.

Tabela 1 - Uso direto da HuggingFace comparado ao uso via LangChain

Conclusão. Para esta aplicação específica, que faz uma única chamada de tradução e nada mais, o uso direto do *transformers* resolve com menos peças e menos coisas para dar errado. O LangChain compensa quando a tradução é uma etapa dentro de um fluxo maior, como gerar um texto, traduzir e depois revisar; quando eu quero poder alternar entre Fake LLM, modelo local e API sem reescrever a aplicação; ou quando vou usar memória, ferramentas e agentes. Foi exatamente esse o caminho das questões 1, 2 e 3 desta parte:

as três aplicações têm a mesma estrutura e mudam basicamente o modelo encadeado.

Parte 2 - Questão 6

Tabela comparativa entre FastAPI puro e LangChain

Antes da tabela, vale uma observação que considero importante: nas cinco aplicações deste TP o FastAPI está sempre presente, porque ele é a camada HTTP. A comparação pedida não é, portanto, entre dois concorrentes que se excluem. O que estou comparando é **FastAPI chamando a biblioteca do modelo diretamente** (questões 1 e 2 da Parte 1, com transformers) contra **FastAPI com o LangChain orquestrando o modelo** (questões 2 e 3 da Parte 2). O LangChain substitui a camada de integração com o modelo, e não o framework web.

Tabela comparativa pelos critérios solicitados

	Facilidade de uso e configuração	Latência e desempenho	Flexibilidade para diferentes modelos	Custo e escalabilidade	Protótipos contra produção
FastAPI puro (com transformers)	Mais simples de começar: menos pacotes e menos conceitos. O caminho do dado é curto e o erro é fácil de localizar. Em troca, cada integração nova é escrita na mão.	Sem camada intermediária e sem rede: a latência é a própria inferência, previsível, mas limitada pela CPU ou GPU da máquina.	Baixa. Trocar o modelo significa reescrever o serviço, porque cada biblioteca tem a sua forma de chamada. Só atende modelos locais do ecossistema escolhido.	Sem custo por chamada; o custo é fixo, de infraestrutura. Escalar replica o modelo na memória, uma cópia por worker, e GPU é caro e fica ocioso em baixa demanda.	Ótimo para protótipo simples e para serviço de um único modelo. Em produção funciona bem como serviço especializado, mas qualquer fluxo mais rico exige código de cola próprio.
LangChain (sobre o FastAPI)	Interface única para todos os modelos (invoke e o operador de composição). Em troca, mais pacotes, mais conceitos (prompt, chain, parser) e uma API que muda rápido entre versões.	A inferência é a mesma; o framework acrescenta um sobrecusto pequeno. Com modelo por API, a rede domina a latência. Cadeia com vários passos soma a latência de cada etapa.	Alta, e é o principal ganho. Fake LLM, HuggingFace local e API da OpenAI são trocados em uma linha, mantendo prompt, parser, rotas e modelos Pydantic intactos.	Depende do provedor escolhido, e é justamente isso que ajuda: permite começar sem custo (Fake LLM ou modelo local) e migrar para API conforme a demanda, ou voltar atrás para controlar custo.	Excelente para protótipo, por causa do Fake LLM, e para produção com fluxo de várias etapas, memória, ferramentas e agentes. É sobrecusto desnecessário se a aplicação faz apenas uma chamada simples.

Tabela 2 - FastAPI puro comparado ao LangChain, por critério

Detalhamento por critério

Facilidade de uso e configuração. Nas questões da Parte 1 eu precisei apenas do FastAPI e do transformers, e o código vai direto ao ponto: monto o pipeline e chamo. Nas questões da Parte 2 entram pacotes e conceitos novos, mas ganho um padrão: as três aplicações com LangChain têm a mesma estrutura, e a diferença entre elas é basicamente qual modelo está encadeado. Como iniciante em Python, achei o caminho direto mais fácil de entender na primeira vez, e o caminho com LangChain mais fácil de repetir depois da terceira vez. Um ponto de atenção real é a velocidade com que a API do LangChain evolui: código escrito hoje pode precisar de ajuste em poucos meses, o que foi comentado em aula.

Latência e desempenho. O LangChain não melhora nem piora de forma significativa a inferência em si, porque quem executa o modelo continua sendo a mesma biblioteca. O que muda o patamar de latência é a *escolha do modelo*: local contra remoto, pequeno contra grande. Na questão 3 da Parte 2, a latência é praticamente a mesma da questão 2 da Parte 1, porque o modelo é do mesmo tipo e roda no mesmo lugar. Na questão 2 da Parte 2 a rede passa a dominar. O sobrecusto do framework só se torna relevante em cadeias longas, onde cada etapa é uma chamada a mais.

Flexibilidade para diferentes modelos. Aqui a diferença é grande e é a principal razão para usar o LangChain. Nas três aplicações da Parte 2, o que muda de uma para a outra é essencialmente a linha que cria o modelo: *FakeListLLM*, *ChatOpenAI* ou *HuggingFacePipeline*. Todo o resto, prompt, parser, rotas, validação e modelos de resposta, permanece. Sem o framework, cada uma dessas trocas seria uma reescrita da camada de serviço. Vale registrar a ressalva da questão 5: essa flexibilidade rende menos com modelos que não entendem instrução, como o opus-mt, porque o prompt deixa de ser um ponto de ajuste.

Custo e escalabilidade. Este critério depende menos do framework e mais de onde o modelo roda, e por isso merece a análise mais objetiva. Com modelo local, o custo é predominantemente fixo: pago o servidor ligado, e o custo adicional de cada requisição é próximo de zero. Com modelo por API, o custo fixo é próximo de zero e o custo variável cresce de forma linear com o número de requisições e com o tamanho dos textos. Chamando de *Cfixo* o custo de manter a infraestrutura no período, de *Cchamada* o custo médio de uma chamada à API e de *n* o número de requisições, o modelo local custa cerca de *Cfixo* e a API custa cerca de *n x Cchamada*. Os dois se igualam em *n* = Cfixo / Cchamada*: abaixo desse volume a API é mais econômica, porque não se paga servidor ocioso; acima dele, hospedar o próprio modelo compensa. Quanto à escalabilidade, as restrições são de naturezas diferentes: no modelo local o limite é hardware, principalmente memória, porque cada worker carrega uma cópia dos pesos; na API o limite é cota e orçamento, e a minha aplicação fica leve e fácil de replicar. A contribuição concreta do LangChain neste critério é indireta, mas útil: como a troca de provedor custa uma linha, fica viável decidir por custo e mudar de estratégia sem reescrever a aplicação, inclusive começar local e migrar para API, ou o contrário, ao cruzar o ponto de equilíbrio.

Adequação para protótipos contra aplicações em produção. Para protótipo, o LangChain tem uma vantagem que o caminho direto não oferece: o Fake LLM. Com ele eu testei a API inteira, com rotas, validação e tratamento de erro, sem baixar modelo, sem gastar cota e com resposta imediata e determinística, como mostrei na questão 1 desta parte. Para produção, a escolha depende da forma da aplicação: se ela faz uma única chamada a um único modelo, o caminho direto é mais enxuto e tem menos peças para falhar; se ela encadeia etapas, usa memória, chama ferramentas externas ou pode precisar trocar de provedor, o LangChain paga o seu custo com folga. Na prática, os dois convivem: o FastAPI continua sendo a porta de entrada, e o LangChain entra quando a orquestração deixa de ser trivial.

Resumo da minha conclusão. Não vejo isso como uma escolha definitiva. Para aprender e para um serviço simples e especializado, FastAPI com a biblioteca do modelo chamada diretamente é mais claro e tem menos partes móveis. A partir do momento em que a aplicação precisa de mais de um modelo, de mais de uma etapa ou da liberdade de trocar de provedor, o LangChain passa a compensar, e o custo de aprendê-lo se paga na primeira troca que eu não preciso reescrever.

Apêndice A

Diagramas de arquitetura das aplicações (Parte 2, questões 1, 2 e 3)

Os diagramas abaixo também estão no arquivo README.md de cada projeto, junto com a tabela de etapas do fluxo de dados. Reproduzo aqui para que este documento fique completo.

A.1 - Questão 1: chatbot com Fake LLM

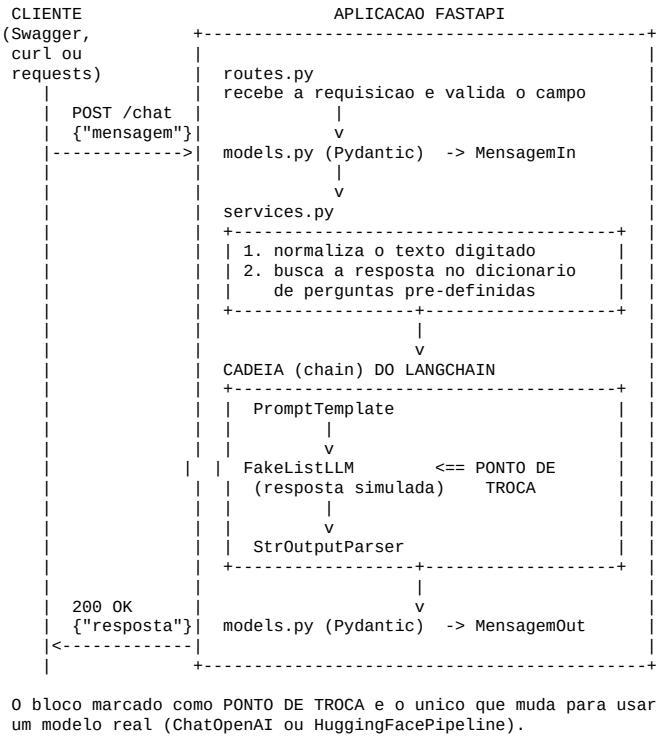


Figura 1 - Arquitetura do chatbot com Fake LLM

A.2 - Questão 2: tradução com a API da OpenAI via LangChain

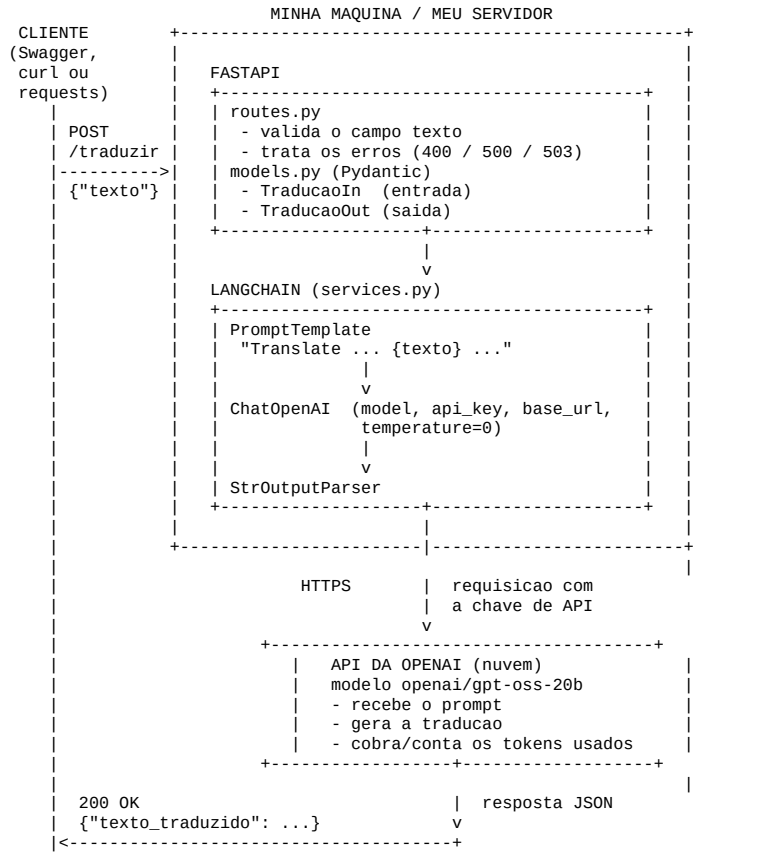
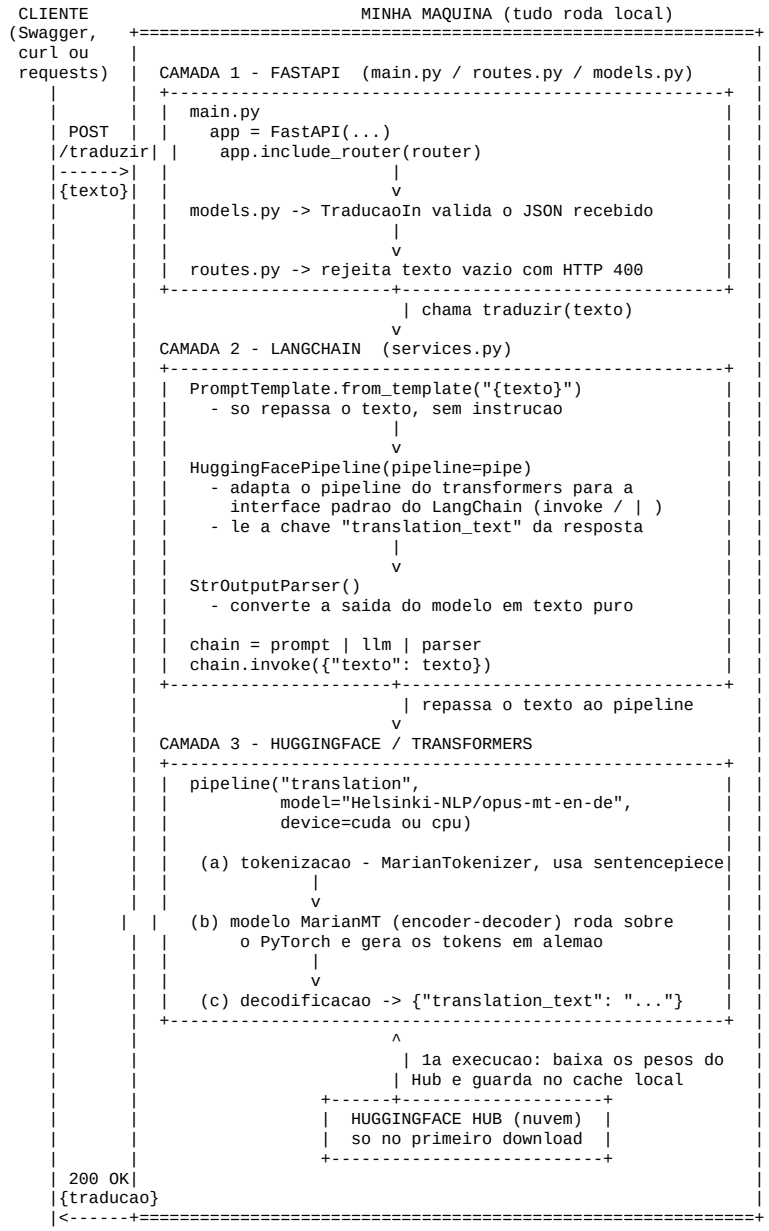


Figura 2 - Arquitetura da tradução com a API da OpenAI via LangChain

A.3 - Questão 3: tradução com HuggingFace via LangChain




```
Interacoes entre os componentes:
  FastAPI     -> LangChain    : chama traduzir(), que executa chain.invoke()
  LangChain   -> HuggingFace  : HuggingFacePipeline repassa o texto ao pipeline
  HuggingFace -> LangChain    : devolve {"translation_text": ...}
  LangChain   -> FastAPI      : StrOutputParser entrega a traducao como string
  FastAPI     -> Cliente      : serializa TraducaoOut e responde em JSON
```

Figura 3 - Arquitetura detalhada da tradução com HuggingFace via LangChain

Apêndice B

Referências

- FastAPI. *Documentação oficial*. Disponível em: fastapi.tiangolo.com
- Hugging Face. *Transformers documentation - Pipelines*. Disponível em: huggingface.co/docs/transformers
- Hugging Face. *Model card: openai-community/gpt2*. Disponível em: huggingface.co/openai-community/gpt2
- Hugging Face. *Model card: Helsinki-NLP/opus-mt-en-fr*. Disponível em: huggingface.co/Helsinki-NLP/opus-mt-en-fr
- Hugging Face. *Model card: Helsinki-NLP/opus-mt-en-de*. Disponível em: huggingface.co/Helsinki-NLP/opus-mt-en-de
- LangChain. *Documentação oficial*. Disponível em: python.langchain.com
- TIEDEMANN, J.; THOTTINGAL, S. *OPUS-MT - Building open translation services for the World*. Proceedings of EAMT, 2020.
- RADFORD, A. et al. *Language Models are Unsupervised Multitask Learners*. OpenAI, 2019.
- Pydantic. *Documentação oficial*. Disponível em: docs.pydantic.dev
- XAVIER, Tiago. *Desenvolvimento de Data-Driven Apps com Python* - slides e aulas da disciplina, etapas 1 a 5. Instituto Infnet, 2026.

Declaração de uso de ferramentas de inteligência artificial

Conforme autorizado no enunciado deste TP, utilizei ferramenta de inteligência artificial como apoio ao desenvolvimento desta atividade.

- **Ferramenta utilizada:** Claude (Anthropic), em novembro de 2026.
- **Em que foi usada:** revisão e organização do texto das respostas discursivas, conferência da estrutura dos projetos, apoio na montagem dos diagramas em modo texto e revisão do código Python.
- **O que foi feito por mim:** a escolha da arquitetura das cinco aplicações, a decisão de separar o código em main, routes, models e services, a definição dos endpoints e dos códigos de status HTTP, os testes de execução e a verificação de todas as afirmações técnicas deste documento.
- **Decisões que posso justificar.** As principais escolhas de implementação estão explicadas no README de cada projeto e nas respostas deste documento, entre elas: a quebra do texto em frases na questão 2 da Parte 1 e o seu efeito colateral na coesão; o uso de temperatura zero na tradução por API; o template contendo apenas a variável de texto na questão 3 da Parte 2, porque o opus-mt traduziria a própria instrução; o cache do pipeline e da cadeia em variável de módulo para não recarregar o modelo a cada requisição; e a decisão de deixar as funções de rota sem async, porque a inferência é bloqueante.

Tenho ciência de que resultados de ferramentas de IA podem ser imprecisos ou tendenciosos. Por isso conferi na documentação oficial as informações técnicas utilizadas, em especial as tarefas aceitas pela classe HuggingFacePipeline do LangChain e o formato da resposta dos pipelines de geração de texto e de tradução do transformers.